SyPy: Python Reference Manual for SynthEyes 2026

©2012-2026 Boris FX, Inc.

The SyPy ("sigh-pie") module allows extensive control of a SynthEyes application process from within a Python programming environment using an object-oriented interface. The SynthEyes scene and user interface can be manipulated directly to add additional functionality, or to interface with other applications to integrate with a studio workflow. SyPy can launch SynthEyes instances, control existing ones, or be run by SynthEyes at the user's request to implement custom tools.

SynthEyes interfaces with a Python interpreter using a client/server approach, much like an X server on Linux. This provides important advantages of flexibility and reliability. You can use your own version of the Python interpreter, or use the Python interpreter built into third-party applications such as Shotgun or Maya. With this approach, you do not have to set up python/python communication channels: you can directly control SynthEyes from another app.

Usage

Per the license agreement, SyPy and other scripting technology may not be used to

- create SynthEyes systems that process shots without artist supervision,
- create replacement user interfaces that allow a SynthEyes license to be remoted or virtualized, or shared between users who are not license holders, or
- create services that process and return shots submitted by remote or third parties through SynthEyes without local artistic supervision.

Support: Script programming requires a substantial degree of expertise and is not covered by our normal support. We can't debug your code nor write code for you. We reserve the right to charge an additional technical support fee to cover scripting.

Prerequisites and Setup

Using SyPy requires a number of elements for the programmer and for the actual run-time environment.

On the machine to be used with SyPy, you need:

- A SynthEyes installation and license
- A Python installation of a compatible Python version of your choice, generally 2.7 or 3.x. (Python is open source)
- Generally, the Python user-interface module of your choice, such as PySide, PyQT, etc

For 2.7 (or 2.6), you should use SyPy as described here. For Python 3, use the SyPy3 library which is also included in SynthEyes.... it's externally the same except you must reference SyPy3 variously.

SynthEyes does not include a specific Python installation, it is "bring your own python" so that you can continue to use the version you already use in your studio. See the section "Configuring Python and SynthEyes" below for setup directions.

What to Read

The SyPy programmer will need to be acquainted with a number of subjects:

- How to program in Python
- The SynthEyes application, ie the SynthEyes User Manual (SynthEyesUM.pdf)
- Sizzle object attributes, as described in the Sizzle manual (SizzleManual.pdf)
- This SyPy API document

The information in the Sizzle manual is necessary because the python interface exposes the same object attributes in python as those in Sizzle; the many attributes are not re-documented here.

SyPy Module Overview

SyPy's main programmer-level interface is called SyLevel; you can think of it as "high level" as it interacts with SynthEyes at a relatively high level, allowing you to open new SynthEyes instances, open shot files, examine trackers and meshes, etc.

Other files include:

- **syobj** - an intermediate-level class used to represent references to SynthEyes shots, trackers, objects, etc
- **sywin** - a class representing windows and controls within SynthEyes
- **symenu** - a class representing menus and submenus
- **sypy** - the low-level interface from python to SynthEyes via the communication channel
- **syconfig** - configuration variables that you might need to adjust, such as the full path name of the SynthEyes application
- **sytrans** - module with the SyTrans class for manipulating transform matrices plus various utility functions
- **sytalker** - a small stand-alone program that allows hand debugging via the communication channel

Note that for Python 3, you'll need to import from SyPy3 not SyPy, though if you want your scripts to work in either environment, you can install SyPy3 as SyPy instead.

Configuring Python and SynthEyes

To run python from SynthEyes, and vice-versa, the SyPy package must be installed into python. The exact location of python depends on which operating system you have. You will need to have administrative permissions to do so.

You can install SyPy into any compatible python interpreter, so that SynthEyes can be accessed directly from inside those other applications. That includes the python interpreter built into other

applications, such as Blender or Cinema 4D, or the python interpreter of your favorite Python IDE package. That's right, you can develop for SyPy directly inside your favorite IDE!

> **NOTE**: The environment variable SYNTHEYES_PYTHON_PATH can contain the name of the final executable (not just the path) for python, and if it is present, it will be used to start python scripts from inside SynthEyes. It overrides any value set in the preferences file.

SyPy/SyPy3 is simply some files that get copied into the right location, a folder named site-packages. See "Installing Python Modules" on the python.org web site for background. You can do this any way that works; we've provided some directions below.

Before installing, you should verify in SyPy's syconfig.py the location that you have installed ***SynthEyes*** in. That location is used when python is started first, if SyPy is asked to start SynthEyes.

> **TIP**: SyPy and SyPy3 are included in every SynthEyes install. Just look in the folder where SynthEyes is installed.

SyPy includes three scripts, installWin.bat, installMac.csh, and installLinux.csh, to help install SyPy. The scripts contain the python path names to be installed to, so you should be sure to edit them before running them, to reflect your python version and install location. For the script to be able to update the python install, you'll probably need to adjust some folder permissions first or run the Mac and Linux scripts as superuser.

Windows

On Windows, you must install Python 2.7 or 3 onto your machine first—Windows does not contain python. (Or you can locate python in one of your other applications.) You can download it from python.org.

If you have changed the SynthEyes or python install location, you will need to update SyPy/SyPy3's syconfig.py.

Copy the folder **C:\Program Files\BorisFX\SynthEyes 2026\SyPy** into **C:\Python27\Lib\site-packages** Windows should ask you for permission to do so.

For python 3, copy the folder **C:\Program Files\Boris FX\SynthEyes 2026\SyPy3** into **C:\Program Files\Python37\Lib\site-packages** Windows should ask you for permission to do so.

Depending on your exact version and installation details, you may have to adjust those folder names.

Start SynthEyes and Edit/Preferences. Go to the System section and make sure that the python executable filename there corresponds to your system. Use **pythonw.exe** if you are using SynthEyes-supplied scripts, or use **python.exe** to display a Windows command window for diagnostic output.

Mac OS X

If you have changed the SynthEyes install location, you will need to update SyPy/SyPy3's syconfig.py.

For python 2.6/2.7, use the Finder to copy the folder **/Applications/BorisFX/SynthEyes 2026/SyPy** into **/Library/Python/2.6/site-packages** OS X should ask you for permission to do so. Alternatively, you can open a terminal window then run the following commands:

```
cd /Applications/SynthEyes/SyPy     # or SyPy3
su                  [or sudo]
csh +x installMac.csh
./installMac.csh
```

Python 3 is not included in macOS by default, so you'll need to install it, either from python.org or using Homebrew ("brew python"). You should then do the process above with the names updated to reflect where you've installed python3.

Start SynthEyes and Edit/Preferences. Go to the System section and make sure that the python executable filename there corresponds to your system.

Linux

If you have changed the SynthEyes install location, you will need to update SyPy's syconfig.py.

Copy the folder **/opt/BorisFX/SynthEyes2026/SyPy** into **/usr/lib/python2.6/site-packages** Linux should ask you for permission to do so. Alternatively, you can open a terminal window then run the following commands:

```
cd /opt/SynthEyes/SyPy
su                  [or sudo]
csh +x installLinux.csh
./installLinux.csh
```

Adjust the filenames as needed for other python versions, including using SyPy3 for python3.

Start SynthEyes and Edit/Preferences. Go to the System section and make sure that the python executable filename there corresponds to your system.

To Other Apps

To install to another app's python, use your operating system's search tools to locate the folder named site-packages within the app (or the python it uses). Copy SyPy or SyPy3 into site-packages, depending on which version that app uses.

You can set up SynthEyes's preferences to launch python within your app, or have it use the system python and use the other app's python only for workflow integration tasks launched within that app, rather than SynthEyes.

Talking to SynthEyes

SyPy's python code talks to a component of SynthEyes called the "listener," which is similar to a small web server built into SynthEyes. It listens for commands from SyPy, and executes them as they are received.

Python calls up the listener via an internet-style port, though since it is a local connection, no external internet access is required. ***Note***: connecting to a SynthEyes process on a different machine is not permitted or possible.

The port number is 16 bits, ranging from 0 to 65535, though per the IANA standard, private ports such as these should be in the 49152 to 65535 range. A "pin" code is also required to provide some defense against any hostile remote browser script trying to talk to any running SynthEyes instance. The pin code is typically a dozen random alphanumeric characters. When SynthEyes is started, it can be told to listen for commands on one or more specific ports via the command line, or it can be told to pick a port at random within a specific range, as set on the preferences.

IMPORTANT: The listener is not so much a security hole as an insecurity superhighway. Any application on your machine---for example a Javascript in a browser window---can commandeer a running SynthEyes instance, even if it does not know the port number to start with (a scan will take only a few seconds). Once controlling SynthEyes, it can cause that instance to do arbitrary damage to your machine, overwriting files or potentially causing arbitrary binary code execution. Customers are responsible for securing their networks against such threats; if that cannot be done, do not activate the Listener.

If you are creating a workflow where your external Python-based application will launch SynthEyes, the SyPy interface needs to supply the port and pin on the command line to SynthEyes, so that the new SynthEyes instance will know where to listen for commands from the (correct) python application. SyPy will create that port and pin automatically if you do not supply them.

You can open a new SynthEyes and prepare to talk to it with (use SyPy3 for python 3):

```
import SyPy
hlev = SyPy.SyLevel()
hlev.OpenSynthEyes()
```

If Python has been started with appropriate "-port NNNN -pin PINCODE" arguments, those will be used. Or, you can supply them directly to the OpenSynthEyes call with hlev.OpenSynthEyes(port, pin). If no other information is available, SyPy will use a randomly-chosen port in the range configured in syconfig.py, and a randomly-selected pin code.

If you are writing python scripts that are started from the SynthEyes Scripts menu, or that talk to an already-running SynthEyes, you can talk to an existing SynthEyes instance, with

```
hlev.OpenExisting(port, pin)
```

In this case, you will need to know the port number and pin string that the SynthEyes instance was started with. SyPy will pull them from the Python command line if none are supplied in the call. Do not allow SyPy to create a random port and pin to talk to an **existing** SynthEyes, since the SynthEyes instance will not be listening for that particular random combination.

Hint: the SynthEyes listener can listen on multiple ports, and have multiple connections open simultaneously on each port. So multiple apps and scripts can all talk to the same SynthEyes instance simultaneously (check out the section on Consistency Control below).

A SynthEyes started and being controlled from an external app can start its own local python script and/or python-based user-interface also. Can you say F...L...E...X...I...B...L...E?

SynthEyes's Python Script Handling

SynthEyes allows Python scripts to be started from the Scripts menu, equivalently to SynthEyes Sizzle scripts. The .py files must be in the SynthEyes system or user script folders. (Note that the system script folder location can be set via the preferences.)

The first line of the Python script is read to extract a tool name, which is the contents of the line following the first semicolon. If there is no semicolon, then the filename is used as the script's title.

When SynthEyes starts the Python script, it supplies port and pin information to Python on the command line, as described above. The Python script being started can do an OpenExisting() call with no arguments to reach back into the calling SynthEyes instance. This makes the Python scripts straightforward. Here's the start of a tool that appears on the Script menu as "Scramble Trackers" (as usual, use SyPy3 here for python3)

```
# ; Scramble Trackers
import SyPy
hlev = SyPy.SyLevel()
hlev.OpenExisting()
# ... use the SyPy interface to do whatever you like ...
```

SynthEyes does not wait for the Python script to complete; the script may launch long-lasting user-interface windows, etc. Scripts should lock SynthEyes, using SyPy, to protect against simultaneous user interactions during operations that should be atomic.

Programming SynthEyes in Python

SyPy has an extensive collection of primitives available to control SynthEyes, see the SyLevel class reference. Several additional topics need to be addressed, beyond what the routines themselves do, to ensure a working application.

Direct Attribute Access

You can access all the shots, all the cameras, all the trackers, etc, using lists supplied by SyLevel, ie Objs(). These are equivalent to the Obj, Trk, etc lists in Sizzle. You can then access and change the attributes of those SynthEyes/Python objects. All attributes are reflected (available) automatically to SyPy. See the Sizzle documentation for a description of all the attributes.

If you use SyObj to examine SynthEyes objects (trackers, meshes, shots, etc) and change their attributes, you need to put such changes inside Begin/Accept calls to create an appropriate Undo block, see the next section.

Animated attributes utilize the current frame on the user interface. To access or change attributes at other times, use SetSzlFrame to set an override, or use the GetF/SetF accessors in SyObj.

Attributes that reference other SynthEyes objects produce and consume SyObj structures, ie the unnecessarily complex `shot.Get("cam")` or the simpler equivalent `shot.cam`, where shot is a SyObj referring to a shot, produces a SyObj referring to the shot's camera.

SyPy can also utilize points, vectors, and transforms. A point is represented as a Python list of two floats, a vector as a list of three floats. A transform is a list of four vectors, each a list of three floats (list-of-lists format). Vectors and transforms are in the user's selected coordinate system setting and rotation ordering, unless they are changed locally in SyPy (see Sizzle Interface Control).

For calculations with transform matrices, see the SyTrans class. It contains a transform list of lists and can perform the usual composition and inversion operations transforms.

Undo

Any python-created operations must interact correctly with the SynthEyes undo subsystem. The programmer should always consider the possibility that the user will hit Undo to any script or workflow operation done via python.

When you simulate user-interface operations using SyWin or the menu calls in SyLevel, undo control is already built into the user interface, because you are simply taking advantage of existing operations.

When you perform operations using direct python access to SynthEyes objects, you need to put such changes inside (SyLevel) Begin/Accept calls, so that the operation can be undone or redone if the user desires. Operations can also be cancelled, causing them to be unrolled, which can be advantageous to make temporary "prospective" changes.

Operations that affect the presentation of data in the SynthEyes user interface generally do not create or require an undo block. For example, changing the current time slider or the flags controlling the appearance of the trackers do not create an undo. You can verify whether an undo is required by trying the operation in SynthEyes, then looking at Edit/Undo to see if an undo block has been created.

Some dialogs are very complex, and may maintain an open Undo block for their entire operation (if they have created and cached extensive amounts of information). In that case, you will be limited to user-interface simulation while the dialog is open as Begin() will be unable to acquire the lock.

Note that SyPy contains some code to monitor your Begin/Accept operations, to attempt to identify problems (and raise an exception) before they cause a crash. If you get an assertion from SynthEyes about "holding" or "!holding," that means that you either have an Accept without a Begin, or a Begin without an Accept, respectively.

Consistency Control

The listener operates asynchronously on one or more threads, one per connection, independent of the main SynthEyes thread. If the listener tries to make changes at the same time as the user (or other listeners), chaos will ensue! That must be avoided. The listener provides tools to help, but they must be systematically used by the client software, and even still, simultaneous monkey-clicking and listener operations may be able to find ways to cause crashes. Normally, the user should be quiescent when the client/listener connection is active.

The `Lock()` operation locks out the user and any other listener connections, so that they cannot interfere. The corresponding `Unlock()` releases the lock so that other operations can occur. `Lock` and `Unlock` can be nested.

The `Begin`/`Accept` pair contains `Lock`()/`Unlock`() calls, so operations that change attributes within a `Begin`/`Accept` pair do **not** need an explicit `Lock`/`Unlock`.

Note that Lock() is made to wait by any in-progress user-initiated operation, which may potentially take an unlimited period of time. Lock() will also have to wait if the user has a mouse-button down, for example to drag something.

You should be sure to encompass all related operations within a Lock/Unlock pair. For example, if you request the number of trackers, then begin doing something to them, you should include the access to the number of trackers in the locked portion, in case the user adds or deletes a tracker simultaneous with the start of your operation. This is less an issue for user-initiated python scripts.

Additional Functionality

SynthEyes has an incredibly deep feature set, so it is likely you will encounter areas that cannot be scripted in Python, either directly or via user-interface simulation. Here are some things you cannot do for sure:

- remote-control large dialogs **without having their UI appear** — the UI will always appear;
- make major changes in the SynthEyes file, ie add, reorder, or delete entities or new kinds of information, except via the API or simulated UI operations;
- change the state of the user interface in ways that do not reflect the underlying scene data;
- create mouse operations in the viewports, ie to move, drag, or click mouse buttons or the scroll wheel; or
- access the shot images themselves.

Operations that are not on this list, and are not apparent in the SyPy documentation, may still be possible, as not all listener capabilities are exposed via user-friendly Python routines.

Please contact us about unsupported features, so that we can see if access can be quickly added with additional low-level Python stubs, or if support can be integrated in a future SynthEyes release.

SyLevel Class Reference

SynthEyes Instance Control

`OpenSynthEyes(port=0, pin="")`	Create a new SynthEyes process and connect to it
`OpenSynthEyesWithSNI(filenm, port=0, pin="")`	Create a new SynthEyes process that will open the given filename, then connect to it. The filename can be a movie, an IFL, or the first image of a sequence.
`OpenExisting(port=0, pin="")`	Connect to an existing SynthEyes process
`OpenSNI(filnm)`	Cause the already-connected SynthEyes to open this file.
`MergeSNI(filnm, uni)`	Merge in the specified SNI file. The uni flag (0/1) is the question about making tracker names unique.
`SaveIfChanged()`	If the scene has changed, pop up the Save/Don't Save/Cancel dialog
`CloseSynthEyes()`	Tells SynthEyes to quit and closes the connection
`QuitSynthEyes()`	Tells SynthEyes to quit, without closing the connection
`Close()`	Close the connection, without affecting the SynthEyes process
`SNIFileName()`	Returns the full File/Save path name
`SetSNIFileName(fnm)`	Set the filename used for File/Save and autosave. Should be a FULL PATH NAME!

Adding New Shots

The following calls either create a new scene and add a shot to it, or add a shot to the existing SynthEyes scene. Both regular or stereo shots are supported. For "New" operations, the previous scene should already be saved, as these operations are unconditional and not undoable.

The filename(s) must have complete path information, and may be a movie, an IFL, or the first image in a sequence.

The aspect ratio argument is strongly suggested, it is for the overall image. If no aspect ratio is supplied, it will be calculated based on square pixels. If the aspect ratio must be changed later, it requires several specialized operations and quite a few calculations, so the correct aspect should be supplied initially if it is available (ie from a shot management system).

These calls return the new (Left, if stereo) shot, or None.

```
NewSceneAndShot(filnm, asp = 0.0)
NewSceneAndStereoShot(Lfilnm, Rfilnm, asp = 0.0)
AddShot(filnm, asp = 0.0)
AddStereoShot(Lfilnm, Rfilnm, asp = 0.0)
NewSceneAndSurveyShot(iflnm, asp = 0.0)
AddSurveyShot(iflnm, asp = 0.0)
```

Export Control

These calls control and/or launch exports of the current SynthEyes scene file. The exportFile must be the full path name to be exported to. The exportType is the export name from the File/Export menu.

`Export(exportType, exportFile)`	
`ConfigureExport(exportType, exportFile)`	
`ExportAgain()`	
`ExportMultiple()`	The desired exports must already be configured!
`FindNewScripts()`	Look for new scripts of any type
`RunScriptFile(filenm)`	Run the given Sizzle/Python/Synthia tool script, given a path and file name. (No return value). Windows: use forward slash or double-backslashing if entering filenames by hand, or python may eat them!

User Interface Controls

The User Interface calls do not have to be inside an Undo block: the user interface is not part of the scene state and does not interact with the Undo system.

`Redraw()`	Redraw the scene. You must do this after making any changes to the scene that don't already have their own Redraw call built in.
`ReloadAll()`	Does a deeper refresh of changed items onto the user interface, such as the title bar. Built into the shot accept calls. Call Redraw after this.
`Frame()`	Returns the current user-visible frame number
`SetFrame(frm)`	Set the current frame number. See also `SetSzlFrame` below.
`View()`	Gets the viewport configuration, a string as shown on the selector.
`SetView(viewnm)`	Set the viewport configuration to the given string
`SetRoomWithView(viewnm)`	Sets the viewport configuration, and records this view configuration as the one to use with this room.
`Room()`	The current room name. Will be null at startup
`SetRoom(roomnm)`	Change the room
`AnimStart()`	Get the starting frame number of the animation playback range.
`SetAnimStart(frm)`	Set the starting frame number of the animation playback range.
`AnimEnd()`	Get the ending frame number of the animation playback range.
`SetAnimEnd(frm)`	Set the ending frame number of the animation playback range.
`Play()`	Begin playback in the SynthEyes interface
`Stop()`	Stop playback in the SynthEyes interface
`IsPlaying()`	Returns 1 if SynthEyes is playing, 0 if stopped.
`PlaybackRate()`	Get the relative playback speed, ie 1.0 for normal, 2 for twice, 0.5 for half, 0.001 for frame by frame. The playback rate is negative when playing in reverse.

`SetPlaybackRate(rate)`	Set the relative playback speed, ie 1.0 for normal, 2 for twice, 0.5 for half, 0.001 for frame by frame. Use a negative number to playback in reverse.

Appearance Flags

`ViewFlags(set=0, clr=0)`	Set and clear various view-control flags (bits), then return the original view flags settings.
`TrackerFlags(set=0, clr=0)`	Set and clear various tracker-appearance flags (bits), then return the original tracker appearance flags.

The details of these bits are subject to change so some experimentation is always a good idea. The following single-bit constants are defined in the SyLevel class; for the function, see the corresponding main and right-click menus.

```
VF_show_meshes
VF_double_buffer
VF_show_3dpts
VF_show_seeds
VF_show_solid
VF_show_trackers
VF_show_tracker_pos
VF_show_tracker_err
VF_show_trails
VF_show_image
VF_only_this                # only trackers from this object
VF_show_shadows
VF_only_sel_spline          # only the selected spline
VF_show_flex
VF_show_curve
VF_enable_prefetch
VF_show_lens_grid
VF_seed_path                # show seed path of obj, cam
VF_texture_alpha            # show alpha of texture, not texture
VF_mesh_hideselect          # hide mesh selection color
VF_show_outline             # outlining for camera solid view
VF_show_toon                # "toon" outline for wireframe

TAFFY_show_alltkname        # show all tracker names
TAFFY_show_tkname           # show supv tracker names
TAFFY_show_seltkname        # show selected tracker names
TAFFY_show_vptnames         # show names in 3D viewports also
TAFFY_show_dot              # show trackers dots for all
TAFFY_only_dot              # show tracker dots for all "done" tk
TAFFY_dots_3D               # as dots in 3D view
TAFFY_dots_persp            # as dots in perspective
TAFFY_alt_color             # use alternate colors
TAFFY_radar                 # radar display included
```

Sizzle Interface Control

These routines control the Sizzle environment created when a python call does something using a Sizzle-based accessor routine. Note that these variables are *local* to a particular listener connection. The "Set" routines all return the previous value of the parameter.

The SzlRecalc flag controls whether or not certain splined attributes are immediately recalculated when the value is changed on a specific frame. If you are interactively moving the seed path, you want the rest of the path to update if you move one frame. But if you are loading 3000 points on the path in one operation, there is no need to constantly recalculate the splines. Disabling recalculation with SetSzlRecalc will save a lot of time. If you disable recalculation, be sure to **explicitly** recalculate anything you have changed using the corresponding recalculation Sizzle call. For example, if you load the camera seed path, do a cam.Call("RecalcSeedPath") call at the end for each affected object. Otherwise unkeyed frames will not be updated to match the new keys. You can tell which attributes are affected by SzlRecalc by looking for the matching recalculation call in the Sizzle Scripting guide.

`SzlFrame()`	Get the SyPy frame number override. This frame number is the one used when animated SynthEyes object attributes are accessed. Equivalent to Sizzle's `frame` variable.
`SetSzlFrame(frm)`	Set the SyPy frame number override, which is used when animated attributes of SynthEyes objects are accessed or changed.
`ClearSzlFrame()`	Clear the frame number override (ie to -1). Further references to animated attributes will refer to their value at the current user-level frame number.
`SzlRecalc()`	State of the immediate recalculation flag. 1=recalculate (default), 0=suspend.
`SetSzlRecalc()`	Sets the immediate recalculation flag. Use to suspend spline recalculation when making many changes to a camera/object seed or solve path, or seed or solve FOV etc. See discussion above.
`AxisMode()`	The effective scene axisMode (0=Z Up, 1=Y Up, 2=Y Up Left), as coming from the scene modified by the sizzle setting below. The axis mode setting changes the value of vectors and transforms that are Sizzle attributes (r/w). Note that the base value is for the scene; the axis mode preference is used only when a new scene is created. To change a scene's axis mode, `hlev.Scene().Set("axisMode", nnn)` within a Begin/Accept.
`SzlAxisMode()`	The current axis mode override setting, or -1 if none.
`SetSzlAxisMode(md)`	Set the local axis mode override to this value. No override if the value is -1
`ClearSzlAxisMode()`	Clears the axis mode override by setting it to -1
`RotOrder()`	Reports the rotation ordering setting from the preferences panel, but modified by any local override
`SzlRotOrder()`	The local override for the rotation ordering
`SetSzlRotOrder(ord)`	Set the local rotation order override, or -1 to clear it.
`ClearSzlRotOrder()`	Clears the rotation order override by setting it to -1

`Conform(vecn, veco)`	Calls Sizzle's Vector.Conform routine, adjusting the rotation angle vector vecn to match rotation angle vector veco, returning the vector result. This is used to handle wraparound issues in exporters.

SynthEyes Entity Access

`MakeObj(ty, idx)`	Makes a SyObj of the given type and index. If the index is less than zero, None will be returned, except if the type is GLOBAL or SCENE for which a negative value is appropriate.
`Active()`	Returns a SyObj for the active tracker host
`SetActive(sob)`	Sets the active tracker host object to the one given. Must be done inside an undo block.
`NumByType(ty)`	Find the number of objects of the given type.
`Scene()`	one Scene SyObj
`Globals()`	one Globals SyObj
`Prefs()`	an iterator that also has getattr/setattr attribute-name access. See the section on Preferences.
`Shots()`	a list of all shots in the scene
`Objects()`	a list of all cameras and moving objects in the scene
`Cameras()`	a list of all cameras in the scene
`Trackers()`	a list of all trackers in the scene
`Meshes()`	a list of all meshes in the scene
`Lights()`	a list of all lights in the scene
`Extras()`	a list of all extra helper points in the scene
`Notes()`	a list of all notes in the scene
`CreateNew(ty)`	Create a new SynthEyes object of the specified type. Use for OBJ, MESH, LIGHT, NOTE. New objects are always moving objects attached to the current open shot. Returns the new object.
`CreateNewTracker(obj)`	Create a new tracker attached to the specified camera or moving object (SyObj), returns the new tracker.
`CreateNewPhase(phty)`	Create a new phase, given its numeric type. Returns the new phase.
`Delete(sob)`	Delete an object, either a camera/moving object, tracker, phase, mesh, extra point, note, or light.
`ClearSelection()`	Unselect, clears single, tracker, and mesh selects
`SelectedTrackers()`	A list of all selected trackers, which are always on the CURRENT object
`SelectedMeshes()`	A list of all selected meshes in the entire scene
`SelectedObject()`	Returns a single-selected object. Could be a single tracker or mesh if only one is selected, or a camera or light, where only one objected can be selected at a time. See `isSelected` attrs
`Select1Object(obj)`	Clear all selections, then select this one object of any selectable type (eg not shots). See `isSelected` attrs

`FindByName(ty, nm)`	Find the object of the given type with the given name. The ty argument can also be CAM for cameras, or TRKCUR for trackers on the currently active object.
`FindObjByName(nm)`	Find a camera or moving object by name
`FindTrackerByName(nm)`	Find a tracker by name. **Beware**: tracker names such as Tracker01 frequently occur on many or all objects! The first match will be returned, regardless of object.
`FindTrackerOnObjectByName(ob, nm)`	Find the tracker on this specific object
`FindMeshByName(nm)`	Find a mesh given its name
`ReadOBJ(fnm)`	Read obj mesh, return a mesh SyObj or None. See also SyObj.ReadObj()

Undo Control

All changes to the SynthEyes ***scene*** must be made inside a Begin/Accept undo block, so that the change can be undone or redone, in any order intermixed with user-initiated operations. Changes to SynObj attributes (.Set, .SetF) must be inside a Begin/Accept pair. Changes to the SynthEyes user interface that are not undoable should not be inside a Begin/Accept, for example, changes to the current frame number or the viewport configuration. Operations that simulate a user interface action, such as clicking a button, should **not** be inside a Begin/Accept because they will already have their own undo support.

See the Special Shot Routines if you are making changes that affects shots and the RAM cache.

`Begin()`	Begin an Undo block and lock SynthEyes UI. Use BeginShotChanges if a shot or image preprocessor will be modified. May not be nested, use InUndo() to tell if needed.
`Accept(titl)`	Complete an Undo block, giving it this title. Unlock and redraw.
`Continue(titl)`	Lock and reenter an existing Accept block of this name, or start a new Undo block if the last one isn't the same name.
`Cancel()`	Cancel and unroll everything done in this undo block. Unlock and redraw.
`InUndo()`	Returns non-zero if in an existing Begin/Accept undo block
`HasChanged()`	Returns 1 if the scene has unsaved changes
`ClearChanged()`	Clears the scene-changed flag
`FlushUndo()`	Flush the undo system, nothing will be undoable or redoable.
`Undo()`	Undo the last begin/accept block, ie Edit/Undo. Includes ReloadAll and Redraw
`Redo()`	Redo the next undone begin/accept block, ie Edit/Redo. Includes ReloadAll and Redraw
`UndoCount()`	Number of undoable operations
`RedoCount()`	Number of redoable operations
`TopUndoName()`	The name of the top undo block, shown as Edit/Undo ...
`TopRedoName()`	The name of the next redo block, shown as Edit/Redo ...

`Lock()`	Lock SynthEyes to group multiple operations together. Included in Begin, so Lock shouldn't normally have to be used directly.
`Unlock()`	Unlock the user interface. Included in Accept, and Cancel.

Special Shot Routines

`BeginShotChanges(sht)`	Use this version to begin an undo block that will modify the settings of a shot, or its image preprocessor settings.
`BeginStereoChanges(lsht, rsht)`	BeginShotChanges for stereo.
`AcceptShotChanges(sht, titl)`	Wrap up changes to a shot or its image preprocessor settings. Builtin redraw and reloadall.
`AcceptStereoChanges(lsht, rsht, titl)`	Wrap up changes to a stereo shot. . Builtin redraw and reloadall.
`PreValidate(sht)`	Creates an undo entry that will cause a shot's RAM cache to be revalidated after an undo. Active ingredient in BeginShotChanges and BeginStereoChanges.
`PostValidate(sht)`	Creates an undo entry that will cause a shot's RAM cache to be revalidated after a redo, and validates the cache immediately. Active ingredient in AcceptShotChanges and Accept StereoChanges.
`Validate(sht)`	Validates all frames in the RAM cache for this shot immediately, making sure they correspond to the current image preprocessor settings, and flushing them if not.
`FlushShot(sht)`	Unconditional flush of all images in the RAM cache for this shot
`SetShotAspect(sht, asp)`	Change the aspect ratio of a shot and update related items. If you do this, you must also include a ShotAspectChanging call in the same undo block. Do **not** use shot.Set("aspect", 3.14159) to change the aspect of a shot!
`ShotAspectChanging(sht, original_asp)`	Use this at the end of an undo block that has included SetShotAspect. It must be given the **original** aspect ratio, in order to adjust the splines correctly
`ShotSingleImageName(sht, frm)`	Retrieve the full path name of the image for a specific frame in an image sequence. Returns the empty string "" if the frame number is bad or the shot is a movie.
`ApplyToTrackers(sht, mod)`	Apply image preprocessor effect to tracker positions, see the Output tab in the image preprocessor and Shot.AdjustTrackers() in the Sizzle manual.
`RenderShot(shot)`	Renders the shot (given by a SyObj), producing an image sequence or movie.

Preferences

Preferences are stored in a separate area. Changes to preferences must be enclosed in BeginPref()/AcceptPref(), though there is no undo for preferences changes. They do not show up in the main application Undo/Redo buttons.

SynthEyes organizes preferences in a big table that is displayed in the left-hand side of the preferences dialog. Python can access preferences according to that table. (There are some other preferences on the right hand side; they currently don't have a direct Python interface.)

You can access the preferences using the methods listed here, or via the Prefs() method, which returns an object that is both an enumerator and variable-property method, ie `hlev.GetPrefFromName("UI font size")` or `hlev.Prefs().Get("fontsize")` or `hlev.Prefs().fontsize`.

`BeginPref()`	Begin making changes to the preferences
`AcceptPref()`	Finish changing preferences.
`Prefs()`	Returns a special preferences object that can serve as an iterator in for loops, and offers attributes for each preference, using its variable name.
`NumPrefs()`	Number of accessible preferences
`PrefVariable(idx)`	A programmer-style variable name for the preferences object, used for Prefs()
`PrefName(idx)`	User-visible preference name
`PrefDescription(idx)`	Tooltip description text
`FindPrefFromVariable(prefnm)`	Get the index from the variable name
`FindPrefFromName(descr)`	Get the index from the user-readable name
`GetPrefFromIndex(idx)`	Get the preference's value from the index (int/float/string)
`GetPrefFromName(nm)`	Get the preference's value from the user name
`GetPrefFromVar(nm)`	Get the preference's value from the variable name. You can also use the pithier `hlev.Prefs().nm`
`SetPrefFromIndex(idx, val)`	Change the preference to the int/float/string value. Must be within a BeginPref/AcceptPref pair.
`SetPrefFromName(nm, val)`	Change the named preference to the int/float/string value. Must be within a BeginPref/AcceptPref pair.
`SetPrefFromVar(nm, val)`	Change the preference (specified by variable name) to the int/float/string value. Must be within a BeginPref/AcceptPref pair. You can also use the pithier `hlev.Prefs().nm = val`
`FolderPref(pnm)`	Returns the full path name for the given folder preference: "BATCHIN", "BATCHOUT", "IMAGE", "SCENE", "IMPORT", "EXPORT", "PREVIEW", "SCRIPT", "PHASELIB"
`SetFolderPref(pnm, fpath)`	Sets the given (`pnm`) folder preference to the full path name `fpath`. The valid `pnm` values are given above.

Menu Operations

These operations let Python "click" various menu operations, as specified by name. The name is given by the bare name, ie "Add Shot", *not* "Shot/Add Shot". The item name must be, and usually is, unique. It will be searched for recursively if it is within a submenu. The Main versions look in the overall main application menu. The Top versions look in the specified top-level menu, ie "File", "Track", etc. The AndWait versions don't return until the operation completes, ie they wait out any required user interaction on any dialogs that pop up. The AndContinue versions return immediately, so any dialog that appears can be driven by remote control from python.

Before operating on menus, you should call InitMenu() to make sure that dynamically-constructed menu items are built. The underlying routine is called within SynthEyes when the user clicks on a menu directly, so you might not encounter a problem by omitting InitMenu until you try to run your script before the user has done anything.

Note: right-click menus are not currently accessible, except via Actions.

`ClickMainMenuAndWait(itemnm)`	the bare name "Add Shot"
`ClickMainMenuAndContinue(itemnm)`	the bare name "Add Shot"
`ClickTopMenuAndWait(topnm, itemnm)`	ie "Shot", "Add Shot"
`ClickTopMenuAndContinue(topnm, itemnm)`	ie "Shot", "Add Shot"
`InitMenu()`	Updates the main menu based on the apps current status
`MainMenu()`	Returns the main menu's SyMenu object
`TopMenu(topnm)`	Returns the SyMenu object for the top-menu with the given name.

Window Operations

You can script operations that simulate user actions, such as clicking buttons, selecting an option, closing windows, etc. Most of this occurs using the SyWin class. SyLevel provides the accessors to get the initial top-level SyWin objects (windows), which you can then examine and manipulate further.

`Main()`	Returns the SyWin for the main window
`Popup()`	Returns the SyWin for the single active modal popup window.
`Status()`	Returns the SyWin for the status line window.
`Floating(wtitl)`	Returns the SyWin for the floating window with the given title. *Implementation limitation:* the Floating() call is unable to locate a floating Camera View (only).

For example, you can click the "Zoom lens" button on the main panel when the Summary panel is open using the following snippet:

```
hlev.Main().ByName("Zoom lens").ClickButtonAndWait()
```

> **Tip:** you can use Synthia to easily look around at the window structure; it provides name, class, and ID information for windows.

Actions

Actions are a short-hand way to locate and control right-click menu items in viewports, buttons on the control panels, and on the main menu. The action is identified by a name string and the underlying id number can be determined as well. You can see the actions on the Edit/Keyboard Manager list, with different actions for the different contexts.

The numeric identifier shows the type of action: numbers under 30000 are button-type actions that need to be "clicked", numbers from 30000 to 40000 are pseudo-menu actions that can be performed at any time, from 40000 to 50000 are main and right-click menu actions, and over

50000 there are dynamically-generated menu IDs (not accessible as actions) corresponding to the most-recently-used files, active objects, scripts, safe areas, phase types, etc.

Actions are the only way to access right-click menus in viewports (Camera View, Perspective View, etc). For example,

```
camvu = hlev.Main().ByClass("CamVu")
camvu.PerformActionByNameAndWait("Cam/Lock Selection")
```

will turn on the Lock Selection item on the camera view's right-click menu. (You may need more complex code to identify the desired viewport window.)

There are also action names for buttons on the various control panels, ie Summary/AUTO. Do not use the Perform* routines for buttons, use something like:

```
idno = hlev.ActionID("Summary/AUTO")            # idno will be < 40000
hlev.Main().ByID(idno).ClickAndContinue()
```

Actions that have no qualifier—no name followed by a slash(/) at the beginning—can generally be sent to the main window at any time, even if the corresponding user-interface element is not presently visible.

Note: there are some generic at-all-times actions from the keyboard manager that have id numbers under 30000 but that should be treated as if they were main menu operations. These special actions can be identified with the context "Any/...", ie you can start a solve at any time with

```
hlev.PerformActionByNameAndContinue("Any/Go!")
```

Action names are automatically generated from the menu information etc, and as such, may change from time to time to improve clarity. The numeric identifiers are unlikely to change. The action names are somewhat more robust than searching the menus or button names directly, because those names might change due to localization.

`Actions()`	Returns an iterator over the entire set of action names (over 600). Use to print out or search them all.
`ActionID(nm)`	Returns the numeric identifier for the action—this is **not** its index into Actions(). If the number is under 40000, it is a button, over 40000, it is a menu-type operation.
`PerformActionByNameAndWait(nm)`	Send the action to the main window to simulate menu operations. Use .ByID().Click...() for button actions.
`PerformActionByNameAndContinue(nm)`	Send the action to the main window, ie to the viewports to simulate menu operations. Use .ByID().Click...() for button actions.
`PerformActionByIDAndWait(idno)`	Send the action to the main window to simulate menu operations. Use .ByID().Click...() for button actions.
`PerformActionByIDAndContinue(idno)`	Send the action to the main to simulate menu operations. Use .ByID().Click...() for button actions.

Utility

`Version()`	SynthEyes version string, eg 2012.09.1020
`Platform()`	PC, LINUX, or MACTEL

`ComputerName()`	The name of the computer SynthEyes is running on.
`UserName()`	The user name of the SynthEyes user.
`AppDir()`	On Windows and Linux, the directory containing the SynthEyes executable. On OS X, the SynthEyes.app directory, which is inside the SynthEyes directory.
`UserDir()`	The user's directory, ie Script/User Script Folder
`MilliSleep(millisec)`	Cause **SynthEyes** to sleep for this many milliseconds (thousandths)
`Beep()`	Make SynthEyes beep, simple low-level test
`Debug(v)`	Turn on or off some low-level tracing of communications with SynthEyes

SyObj Class Reference

The SyObj class is used to represent (a reference to) SynthEyes objects within the Python environment. Valid types include "TRK", "OBJ", "SHOT", "PHASE", "MESH", "LIGHT", "NOTE", "SPLINE", "LIVEPREP", "STOREPREP", "STER", "EXTRA", "GLOBAL", "SCENE".

A major goal to SyObj is to continue to refer to the same object within the SynthEyes environment even if the object is renamed or other objects are deleted within the SynthEyes environment, without notification to Python. To accomplish that, the SyObj uses a unique identifier supplied by SynthEyes that is a creation timestamp. SyObj references become invalid when a different SynthEyes scene is loaded, or if you close their owning hlev object.

For example, a reference to Tracker01 continues to refer to it if its name is changed to Tracker101. If a new scene is opened in SynthEyes, the reference to Tracker101 is now invalid, even if the new scene now contains a Tracker101, since it is the unique identifier that matters.

SyObj instances are created using indices, which is how SynthEyes keeps track of shots, objects, trackers, meshes etc internally and in the communication path, but those indices are converted to unique identifiers for permanent storage. When the index is required at a later time, it is re-determined at that time.

Attribute values can be integers, floats, strings, SyTrans transforms, points (length 2 lists), vectors (length 3 lists), transform lists (length 4 lists of 3 element lists), or SyObjs. When assigning to an attribute, or calling a function, you **must** supply a value of the appropriate type. Don't pass the string "10" where a numeric argument is expected, for example.

With a SyObj, you can determine or change the value of any of its Sizzle attributes, either directly or via the Get() function; for example, `trk.isEnabled` or `trk.Get("isEnabled")`. Using the Get function, an attribute name can contain multiple parts, such as `cam.Get("rowP.x")` or `trk.Get("cam.nm")`, to access subattributes easily and directly.

Important: While you can access nested attributes, you cannot change nested data elements directly, ie `cam.Get("trans.rowP.x")` is OK, and `cam.Get("shot.cam")` is OK but silly, `cam.Set("trans.rowP.x", 0)` will crash (assign a vector to "trans.rowP").

Animated attributes reflect the value on the current user-level frame, or you can use `SyLevel.SetSzlFrame(frm)` to supply a different frame number temporarily (equivalent to the

`frame` variable in Sizzle). That is typically done when examining a whole series of frames, so that it can be done quickly and silently without having to repeatedly redraw the entire SynthEyes user interface.

`__init__(hlev, type, index)`	Create an object. The SyLevel object (hlev) must be supplied as the owner. The type is "TRK", "MESH", etc The index is as-used by SynthEyes, ie zero-based (whereas Sizzle indices are 1-based for convenience).
`Type()`	The object's type, "TRK", "MESH", etc
`Index()`	The object's index within objects of this type
`UniqID()`	The objects' unique identifier.
`Name()`	The object's current name within SynthEyes
`SetName(nm)`	Change the object's name. Must be within a Begin/Accept Undo block!
`Get(attrnm)`	Get the attribute's value (on the current or overridden frame, if it is animated). The attribute name can have multiple parts, ie the scalar "rowP.x" of a transform. You can also write `obj.Get("attr")` as the pithier `obj.attr`
`GetF(frm, attrnm)`	Get the attribute's value on the specified frame.
`Set(attrnm)`	Change the attribute's value (on the current or overridden frame, if animated). Must be within a Begin/Accept Undo block! Use the special methods for changing the aspect ratio of a shot, do not use this. You can also write `obj.Set("attr", val)` as the pithier `obj.attr = val`
`SetF(frm, attrnm, val)`	Change the attribute's value on the specified frame. Must be within a Begin/Accept Undo block!
`Call(funcnm, *args)`	Call the specified function on this object, where *args is any number of arguments. Must be within an Undo block if the function changes the scene. See Calling Arbitrary Sizzle Functions for more information.
`MovingObjects()`	a list of all moving objects on this camera
`Trackers()`	Returns the trackers attached to this object, which must be a camera or moving object (type "OBJ")
`Objects()`	The objects on this shot
`Meshes()`	Returns the meshes parented to this object
`NewTracker()`	Creates a new tracker parented to this object
`ReadObj(meshfilnm)`	Reloads this (existing) mesh from the given OBJ file. Returns True or False accordingly.

SyPrefObj Reference

The SyLevel.Prefs() iterator produces instances of SyPrefObj. These objects offer a similar interface to SyObj, allowing the preference's value to be accessed or changed. The index cited in the constructor is the index into the preferences panel array.

The Prefs() object can also be used to access the preferences using the variable name, ie `hlev.Prefs().autosave`

`__init__(hlev, index)`	Create a preference object from its UI-table index
`Type()`	Returns "PREF"
`Index()`	the UI-table index
`Name()`	the user-visible name of the preference
`Variable()`	the variable name
`Description()`	the tooltip text
`Get()`	Get the preference's current value
`Set(val)`	Change the preference's value. Must be within a BeginPref/AcceptPref block!

SyTrans Reference

The position and orientation of objects in 3-D for computer graphics and robotics are typically represented as a 3x4 transformation matrix; such matrices can be multiplied to represent compound manipulations. SyPy stores a transform matrix as a list of four vectors, each a list of 3 floats. Since lists are built into python and easy to create and display, transform matrices are supplied as lists of lists from `SyObj.Get()`, for example,

```
>>> cam = hlev.FindObjByName("Camera01")
>>> cam.trans
[[1.0, 0.0, 0.0], [0.0, 0.0, 1.0], [0.0, -1.0, 0.0], [0.0, -100.0, 10.0]]
```

> **IMPORTANT PYTHON 3 GOTCHA**: SyPy3 for Python3 has some additional functionality built around the Get function, the implicit SyTrans.attrib form, and .Call(), which makes it easier to do transform operations that match up with SynthEyes without having to duplicate that functionality in Python. (Especially access to the conversions to rotation angles. See also SyLevel.Conform()) As part of that, whenever a transform value is brought back from SynthEyes, it **appears in Python3 as a SyTrans, not as a list-of-lists as in Python2**. You must add a ".m" to places where the list-of-lists form is needed for your own explicit operations. You can set attributes using either the list-of-lists or SyTrans formats. To create SyTrans use SyPy3.SyTrans(m_data, hlev).

If you want to create and manipulate transform matrices, the sytrans module is available to help. You can use your favorite other package as long as you convert to the list-of-lists format to supply to SyPy.

The SyTrans class in sytrans is a carrier for the list-of-lists data. Here we fetch the camera position, convert it to a SyTrans using the abbreviated SyT function, and then print its m file, which contains the list-of-lists data.

```
>>> defpos = SyPy.SyT(cam.trans)
>>> defpos
<SyPy.sytrans.SyTrans instance at 0x000000000214CBC8>
>>> defpos.m
[[1.0, 0.0, 0.0], [0.0, 0.0, 1.0], [0.0, -1.0, 0.0], [0.0, -100.0, 10.0]]
```

While the Get routine always supplies lists-of-lists(SyPy) or SyTrans(SyPy3) data, the Set routine accepts either lists-of-lists or a SyTrans.

```
>>> hlev.Begin()
>>> cam.Set("trans", defpos)
>>> cam.trans = defpos
>>> cam.trans = defpos.m
```

```
>>> cam.Set("trans", defpos.m)
>>> hlev.Accept('Four different ways to do the same nothing')
```

The SyTrans objects can be created or combined together using the following functions. Notice that they are supplied as both functions that change an existing SyTrans, and as stand-alone functions that return a list-of-lists. The (transform) arguments to the functions can be either a SyTrans or a list-of-lists.

The XRot, YRot, and ZRot functions are particularly convenient for creating transformation matrices that match the specific ordering of rotations from other applications.

Member Functions of SyTrans

`Get(attrname)`	Python 3 only! Returns the named Sizzle attribute of the transform. You can also say xf.rowP. For convenience, .m (specially!) is defined to be the list-of-list form.
`Call(funcnm, rest*)`	Calls the funcnm function using the specified arguments, especially "*" with a single SyTrans argument, to multiply the two together (in Sizzle ordering)
`Pos()`	Returns the position (translational portion) of the transform, ie its fourth column.
`SetPos(pvec)`	Sets the position (translational portion), ie fourth column, of the tranform
`DePos()`	Removes the position portion of the transform (sets it to zero)
`DeRot()`	Removes the rotational portion of the transform, setting it to an identity matrix
`SetProd(a,b)`	Sets the transform matrix to the product of a and b.
`SetInv(a)`	Sets the transform matrix to the inverse of a. Note that a must be an orthonormal matrix (no scaling or skew).
`Prnt()`	Prints the matrix in an orderly format

Standalone functions

See also the SyLevel.Conform routine.

`Prod(a, b)`	Returns the product of a (SyTrans or list-of-lists) and b (SyTrans, list-of-lists, or vector list) as a list-of-lists.
`MapDir(a, b)`	Returns the product of a (SyTrans or list-of-lists) and b (vector list only), **ignoring the translational portion of a**. This is useful when b is a direction, instead of a position.
`Inv(a)`	Returns the inverse of a (SyTrans or list-of-lists). The transform a must be orthonormal, with no scaling or skew.
`Prnt(a)`	Prints out matrix a neatly
`Iden()`	Returns an identity matrix as a list-of-lists.
`XRot(ang)`	Creates a list-of-lists transform that corresponds to a rotation around the positive X axis using the right-hand-rule. The angle (ang) is in **degrees**.
`YRot(ang)`	Create a list-of-lists transform that corresponds to a rotation around the positive Y axis axis using the right-hand-rule. The angle (ang) is in **degrees**.

`ZRot(ang)`	Create a list-of-lists transform that corresponds to a rotation around the positive Z axis axis using the right-hand-rule. The angle (ang) is in **degrees**.
`Cross(a,b)`	Returns the cross product of vectors a and b.
`Norm(a)`	Returns a normalized version of vector a
`FOV2FLen(fov, pwidth)`	Returns the focal length for a given angle of view (fov, in degrees) for a given plate width. The focal length is in the same units as the plate width.
`FLen2FOV(flen, pwidth)`	Returns the angle of view (FOV) corresponding to a given focal length and plate width. The focal length and plate width must have the same (arbitrary) units.

SyWin Reference

The SyWin class encapsulates windows and controls within SynthEyes, so that you can click buttons, select options, etc, operating the user interface by remote control. Wherever possible, you should try to directly access and change the scene through SyObj accessors, rather than trying to make the user interface dance around like a crazy puppet.

SyWin objects are created using Main(), Popup(), and Floating() in SyLevel, which access top-level windows. The methods of SyWin let you locate the subwindows you need to control SynthEyes. By design, these functions allow you to control what SynthEyes does, what a user can do, not do arbitrary things to the SynthEyes windows that the underlying program will not support.

You may find the following piece of Python code helpful (or call `hlev.Main().Dump()`):

```
for w in hlev.Main().BreadthFirst():
      print w.Name(), w.Class(), w.ID()
```

It prints out the name, window class, and ID of every window in SynthEyes's main window, using recursive descent. While many windows can be reliably picked out by name, sometimes you may need to identify the ID once by hand, then use ByID to locate it in code.

Important: Be aware that SyWin window references can become invalid, typically when the window is closed, the panel containing it is changed, or the viewport configuration has changed. If this is a possibility, use IsValid() to verify the window reference. Operating on invalid windows may crash SynthEyes, especially on Mac OS X.

Some routines come in AndWait and AndContinue versions:

If an operation is short and does not open additional dialogs, or the user must handle any additional operations, you can use the AndWait version. The python interpreter will be locked until the ...AndWait routine finishes.

Use the AndContinue version when python must not be locked, so that you can execute additional code to script out additional dialogs that appear. Be aware that your next python operation might occur before the previously-launched operation has had a chance to be effective.

Usage note: When using SyWin, you are controlling values with user values. For example, angles are in degrees, as are SyObj attributes. In the scene data store, values are in radians. Accessed through SyWin, position coordinates will have been swizzled to the user-selected axis ordering, whereas SyObj values are in the user's ordering unless it has been overridden to some other ordering, and internally the scene store is always Z-up.

Debugging note: popup windows are minimized when another application is active. If you are running a python interpreter by hand, you may need to put a small `time.sleep` in front of windows commands and re-activate the SynthEyes application by hand before the command activates, so that the popups are in the proper state.

`Name()`	The name of any kind of window, ie the text on the title bar, in a button, a spinner, an edit control, a selector, etc
`SetName(newnm)`	Changes the name or text value of the window to the specified name. **DANGER**: Use with extreme care. Changing the value of a spinner or edit field this way will not activate the portions of the program that normally respond to such changes, so the control may show the new value without the underlying scene being affected. Whenever possible, use a SyObj.Set(), SetOption, SetSpinner, etc to make a change.
`ID()`	The window ID of a control, a small integer. This is the best identifier for a window, if it has one.
`Class()`	The class name of the window. Note that classes may differ between PC and Mac, especially DIALOG vs #32770
`HWND()`	Return the window identifier (16 character string)
`IsValid()`	Returns 1 if this SyWin is an actual still-alive window
`IsEnabled()`	Returns 1 if the window is enabled. There is no SetEnabled by design!
`Parent()`	The window's parent
`FirstChild()`	The window's first child, if any
`Next()`	The window's next sibling, which can be used to enumerate all the windows at a given level, used in Children()
`Child(nth)`	The nth child of this window. Use IsNull to stop
`Children()`	An enumerator for all the window's children
`BreadthFirst()`	Returns a breadth-first enumeration of this window and all its children as a list. This window is the first element.
`DepthFirst()`	Returns a depth-first enumeration of this window and all its children as a list. This window is the last element.
`ByClass(cls)`	Find the first child window with this class. Recursive.
`ByName(nm)`	Find the first child window with this name. Recursive.
`ByID(id)`	Find the first child window with this ID. Recursive.
`CloseAndWait()`	Close this window, wait out any required actions to do that, including new dialogs being closed, etc. Do not call Close on dialogs that have no close button on their title bar, click the appropriate button instead.
`CloseAndContinue()`	Start closing this window, return immediately so Python has a chance to control any subsequent dialogs directly.

	Do not call Close on dialogs that have no close button on their title bar, click the appropriate button instead.
`ClickAndWait()`	Click this button, wait for any resulting activities to finish. Useful for plain buttons, checkboxes, radio buttons.
`ClickAndContinue()`	Click this button, continue immediately.
`PerformActionByNameAndWait(nm)`	Send the action to this window (a viewport) to simulate right-click menu operations. Use .ByID().Click...() for button actions.
`PerformActionByNameAndContinue(nm)`	Send the action to this window, ie to the viewports to simulate right-click menu operations. Use .ByID().Click...() for button actions.
`PerformActionByIDAndWait(idno)`	Send the action to this window (a viewport) to simulate right-click menu operations. Use .ByID().Click...() for button actions.
`PerformActionByIDAndContinue(idno)`	Send the action to this window (a viewport) to simulate right-click menu operations. Use .ByID().Click...() for button actions.
`IsChecked()`	Returns 0/1/2 as this checkbox, radiobutton, or button is unchecked/checked/indeterminate.
`SetChecked(val)`	Set the checkbox or radiobutton to the given state by clicking it if necessary.
`OptionCount()`	For a dropdown selector or multi-option button, the number of options.
`OptionText(n)`	The text of the n'th choice of a dropdown selector.
`OptionNo()`	The currently-selected option number of a dropdown selector or button or image preprocessor tab selector.
`Option()`	The text of the currently-selected option
`SetOptionNo(n)`	Select the given option number of a dropdown selector or multi-option button or image preprocessor tab selector.
`SetOption(txt)`	Select the option with this text
`EditValue()`	Text value of an editor (Edit, REdit) field, equivalent to Name()
`SetEditValue(txt)`	Change the text value of an editor field, notifying the rest of SynthEyes about it correctly. Preferably, use a SyObj attribute instead.
`SpnValue()`	The numeric value of a spinner. Note that textual labels are not part of the spinner, they are separate windows. So you will need to identify spinners originally by value and record the ID. Preferably, use a SyObj attribute instead.
`SetSpnValue(val)`	Set the spinner's value and notify the rest of SynthEyes correctly. The value is clipped against the spinner's acceptable range. Always use SyObj attributes to change values when possible.
`Dump()`	Prints a recursive breadth-first dump of this window's children.

SyMenu Class Reference

The SyMenu class represents a menu, including the main top menu, any of the top-level menus such as "File", "Track", "Help", etc, or any submenu below that. See also Menu Operations in the SyLevel Class Reference.

> **WARNING:** Python menu objects are represented as a list of submenu operations, for example: start at the top, go into" Script", then go into "Script bar". The menu is searched upon each use. This is a good approach since menus are frequently rebuilt and the physical menu storage may change even if it is apparently the same menu. The representation is problematic when a submenu name changes or if there are multiple submenus with the same name. In that case you can recreate the object, or build it with SubMenuByPos originally.

The menu operations are designed to work with SynthEyes existing menu operations. SynthEyes can rebuild menus at any time; if you aren't careful your new submenus or menu items will be deleted as part of a rebuild, or worse, your additions may compromise SynthEyes's ability to locate the standard items, resulting in malfunction.

Never delete, check, or gray a submenu or menu item you didn't create!

Do not try to check or disable existing SynthEyes menu items, or try to click grayed-out menu items. Attempting to do so will nearly always create problems (which may not be immediately obvious) and will not work as expected. Use Click operations instead, which emulate valid user actions.

For example, you might have something like this:

```
nu = hlev.MainMenu().AddSubMenu(hlev.MainMenu().Count()+1, "Shotgun")
nu.AddMenuItem(0, "Test1", 997)
nu.AddMenuItem(1, "Test2", 998)
nu.AddMenuItem(2, "Test3", 999)
```

To be usable, the example command identifiers 997-999 must be replaced with live numbers taken from other menu items, for example those assigned to some dummy script entries buried deep in the script menu.

`Count()`	The number of items in the menu
`SubMenuByName(nm)`	This menu's submenu with the given name, as evaluated at the time of use (more robust).
`SubMenuByPos(idx)`	This menu's submenu at the given index, as evaluated at the time of use (if needed).
`Exists()`	Returns true if this submenu actually exists at present.
`HasSubMenu(idx)`	Returns true if there is a submenu at this index within the menu.
`AddSubMenu(idx, nm)`	Add the named submenu at the given index. Extreme care must be taken to avoid interfering with SynthEyes's dynamic menu-building activities.
`AddMenuItem(idx, nm, cmdid)`	Add the named menu item to the submenu at the given index. Extreme care must be taken to avoid interfering with SynthEyes's dynamic menu-building activities. When clicked, the menu item will trigger the action determined by

	the given cmdid, which must almost always be obtained by reading it from some existing menu item!
`DeleteByID(nm)`	Delete the item with this command ID.
`DeleteByPos(idx)`	Delete the item/submenu at this index.
`PosByName(nm)`	Return the index of the item with the given name in this menu, or -1.
`IDByName(nm)`	Return the command ID of the menu item with this name.
`IDByPos(idx)`	Return the command ID of the menu item at this index.
`NameByPos(idx)`	Return the item or submenu name at the given index.
`NameByID(cmdid)`	Return the name of the item with the given command id.
`SetNameByPos(idx, nm)`	Sets the item or submenu name at the given index.
`SetNameByID(cmdid, nm)`	Sets the name of the item with the given command id.
`IsCheckedByPos(idx)`	Returns true if the item at this index is checked.
`IsCheckedByID(cmdid)`	Returns true if the item with this command id is checked.
`SetCheckedByPos(idx, v)`	If v is nonzero, check the menu item at this index. **Change only your own menu items, never SynthEyes's!**
`SetCheckedByID(cmdid, v)`	If v is nonzero, check the menu item with this command id. **Change only your own menu items, never SynthEyes's!**
`IsGrayedByPos(idx)`	Returns true if the item at this index is grayed, ie unclickable. **Don't click grayed items!**
`IsGrayedByID(cmdid)`	Returns true if the item with this command id is grayed, ie unclickable. **Don't click grayed items!**
`SetGrayedByPos(idx, v)`	If v is nonzero, gray (disable0 the menu item at this index. **Change only your own menu items, never SynthEyes's!**
`SetGrayedByID(cmdid, v)`	If v is nonzero, gray (disable) the menu item with this command id. **Change only your own menu items, never SynthEyes's!**

SyPy Class Reference

The SyPy object is a low-level "core" object used by the SyLevel object. SyPy contains and controls the detailed low-level communications with the SynthEyes listener thread.

`__init__()`	Constructor, no arguments
`Open(port=0, pin="")`	Open a path to an existing SynthEyes instance
`OK()`	True/False reflecting the status of the connection
`Close()`	Close the connection

Calling Arbitrary Sizzle Functions

You can easily call any function using obj.Call("function", args....). For example,

```
msh.Call("ApplyCameraMap", cam)
dist = msh.Call("IntersectDistance", pos, [1,0,0])  # from vector pos along X
```

The argument-passing and return interface accommodates the python object None, strings, numbers, points, vectors, and transforms, as well as SynthEyes trackers, cameras/moving objects, phases, meshes, splines, lights, extra points, and notes.

Be sure to pass arguments with the correct type, ie don't pass the *string* "10" to an argument expecting a number. You must convert it to the correct type first, or the call will fail.

You'll receive a python object representing the return value, with any of the types above. Note that different python objects can represent the same Sizzle object.

Examples

Here are a few python examples. You can also look for python-based scripts in the system scripting area, such as the Rolling Shutter Analysis (rollingcalc.py).

Changing a Tracked Camera to a Moving Object Shot

It's fairly common for users to track for a while, then realize that they should have set up the shot as a moving-object shot. There's even a tutorial on converting to a camera to an object track.

The following is a python script to quickly do that for us. (Again, reference SyPy3 below when using Python 3):

```
# ; Quick convert camera-track to object-track

import SyPy

hlev = SyPy.SyLevel()
hlev.OpenExisting()

# Go to the solver panel and reset the camera

hlev.SetRoom("Solver")
bid = hlev.ActionID("Solve/Clear")
hlev.Main().ByID(bid).ClickAndWait()      # it's on the Summary panel

# Start our undo block for low-level changes

hlev.Begin()

# Make a new moving object on the active camera
mob = hlev.CreateNew("OBJ")

# Move all the trackers to the new object

cam = hlev.Active().cam                   # active cam for sure
for t in cam.Trackers():
    t.obj = mob

# Make the camera disabled, and the new object active

cam.mode = "Disabled"
hlev.SetActive(mob)
hlev.ReloadAll()
hlev.Accept("Convert to moving object")

# All done... could re-solve here if desired.
```

This script can be run directly, using Script/Run Script, or if it is placed in the system or user script directory, it can be run from the Script menu as "Script/Quick convert camera-track to object-track"

Loading a Shot and Path

This example reads a file that contains the file name of the images for a shot, a number of typical shot setup parameters, and then reads camera path information from that file and loads it into the camera seed path. Then it tracks and solves the shot.

There's a main function that can be called with command line arguments via a small stub at the end. The function can connect to an existing SynthEyes or create a new one. There's a fair amount of debugging printout available if you want; with verbose = 2 you'll see the entire communications between python and SynthEyes.

The incoming rotation angles must be interpreted correctly. The code shows cases for x, y, and z angles in the most common ZXY ordering, or for pan/tilt/roll angles. You'll see that you can easily rearrange to other formats that you might need. You can also readjust for different coordinate axes if needed: this code assumes the incoming data is in Z-Up format.

The recalculation controls are used to prevent spline recalculations from taking too much time; after the path is read, the spline recalculation routine is called directly.

The code shows different ways to accomplish some operations, ie directly changing the scene file or remote-controlling the user interface. Generally the scene access is preferred since it can be done more quickly, without requiring specific UI panels or dialogs to be opened.

You can generate sample input data by exporting a solved shot using the Plain Camera export and adding the initial information in a text editor.

```
#####################################################################
# Scripted open of a new shot by reading a text file with the
# image name, aspect, and FOV and a recorded mocap camera path
#                  Boris FX, Inc.

import sys
import SyPy
from SyPy import sytrans

verbose = 1                    # set the debug level to 0/1/2

hlev = SyPy.SyLevel()

# Call with the descriptive-file filename.
#  Supply valid port/pin to connect to an existing instance,
#  otherwise a new SynthEyes will be created.

def OpenSceneFromFile(descrfnm, port=0, pin=""):
    global hlev, verbose

# Open the description file

    istrm = open(descrfnm, "r")
```

```
# First line is the shot name

    imgfilnm = istrm.readline().strip()

# Second has focal length and H/V plate sizes

    wds = istrm.readline().split()
    rate = float(wds[0])
    focal = float(wds[1])
    plateWidth = float(wds[2])
    plateHeight = float(wds[3])
    if verbose > 0:
      print "rate", rate, "focal", focal, "width", plateWidth, \
            "height", plateHeight
    aspect = plateWidth / plateHeight
    if verbose > 0:
      print "aspect", aspect

# Create a new SynthEyes instance. We could pass it the file name
# to open directly but do it in two steps to show how to do it in
# case we want to run an existing instance

    if port != 0:
      rv = hlev.OpenExisting(port, pin)
      if verbose > 0:
          print "port", port, "pin", pin
    else:
      rv = hlev.OpenSynthEyes()
      if verbose > 0:
          print "OpenSynthEyes returned", rv
          print "port", SyPy.syconfig.DefaultPort(),  \
                "pin", SyPy.syconfig.DefaultPin()
    if not rv:
      if verbose > 0:
          print "couldn't connect to SynthEyes"
      return False
    if verbose > 0:
      print "Connected..."
    if verbose > 1:
      hlev.Debug(1)                 # watch communications...

# Open the shot

    rv = hlev.NewSceneAndShot(imgfilnm, aspect)
    if rv == None:
      if verbose > 0:
          print "couldn't open the shot"
      return False
    if verbose > 0:
      print "Opened shot..."

# Make sure in Z-up mode, override user temporarily

    hlev.SetSzlAxisMode(0)

# Locate the camera and shot (via Sizzle attribute access)
```

```
    cam = hlev.FindObjByName("Camera01")
    sht = cam.Get("shot")

# Adjust the shot parameters more here.... BeginShotChanges not needed
# just to set the plateWidth, but use it in case more complex things
# are added later.

    hlev.BeginShotChanges(sht)
    sht.Set("backPlateWidth", plateWidth) # in mm per Sizzle doc
      # no need to set height, SynthEyes calculates it from the aspect

# Start reading data lines from the file

    frm = 0
    hlev.SetSzlRecalc(0)      # suppress spline recalculation for mass reload
    while True:
      line = istrm.readline()
      if not line:
          break

#    Pull out the tokens

      wds = line.split()
      if len(wds) == 0:
          continue
      elif not wds[0][0].isdigit():
          continue            # header line
      elif len(wds) < 6:
          raise Exception("incomplete data")
      px = float(wds[0])
      py = float(wds[1])
      pz = float(wds[2])
      rx = pan = float(wds[3])
      ry = tilt = float(wds[4])
      rz = roll = float(wds[5])

#     Crank up the right orientation matrix --- for Z-Up coordinates
#     Rotate cam up to pointing along Y, roll around Y, tilt u/d around X,
#       pan around Z

      pose = sytrans.SyTrans()
      if True:    # rx/ry/rz in ZXY order
          pose.SetProd( sytrans.Prod(     \
                            sytrans.YRot(ry),         \
                            sytrans.XRot(rx)),  \
                        sytrans.ZRot(rz))
      else:       # pan/tilt/roll
          pose.SetProd( sytrans.Prod(     \
                            sytrans.ZRot(pan),        \
                            sytrans.XRot(tilt)),      \
                          sytrans.Prod(           \
                            sytrans.YRot(-roll),      \
                            sytrans.XRot(90)))
      pose.SetPos([px,py,pz])

#     Store the data
```

```
      hlev.SetSzlFrame(frm)
      cam.Set("trans", pose)
      frm += 1

# Reenable spline calculations and recalculate now that changes done

    cam.Call("RecalcSeedPath")      # now the deferred recalculation
    hlev.SetSzlRecalc(1)      # turn back on

    hlev.ClearSzlFrame()
    if verbose > 0:
      print "read", frm, "frames"         # TODO

# Set up Known with Estimate lens mode.
# Note that if the scene was already solved, we'd have to use seedFOV
# so we'd have to convert the FL to a FOV with sytrans.FLen2FOV
# If a zooming FOV estimate was supplied, we'd probably want to solve
# once as a Known lens, then do a Refine cycle with the lens set to Zoom

    sht.Set("rate", rate)
    if focal != 0:
      cam.SetF(0, "fl", focal)      # animated, be sure at frame zero
      cam.Set("lensMode", 3)  # known with estimate

# Set from seed path solving mode, as long as we have one

    if frm >= 2:
      cam.Set("mode", "From Path")
      # or a different way....
      # hlev.SetRoom("Solver")            # to Summary room
      # hlev.Main().ByID(1274).SetOption("From Path")
      # or this: hlev.Main().ByID(1274).SetOptionNo(9)

# wrap up a little

    hlev.AcceptShotChanges(sht, "Set up shot")
    istrm.close()

# Launch a solve

    hlev.ClickMainMenuAndWait("Summary")  # to Summary room, for variety
    # hlev.SetRoom("Summary")             # normally this...
    hlev.Main().ByName("AUTO").ClickAndContinue();    # or via Action code

    if verbose > 0:
      print "Success!"
    return True

#-----------------------------------------------------------------------
# Make it work as a standalone script with description filename on the
#     command line

if __name__ == "__main__" and len(sys.argv) >= 2:
    if verbose > 0:
      print "source file name:", sys.argv[0]
    if len(sys.argv) >= 4:          # port and pin
```

```
      OpenSceneFromFile(sys.argv[1], int(sys.argv[2]), sys.argv[3])
    else:
      OpenSceneFromFile(sys.argv[1])
```

Creating a Mesh

The following code creates a small planar mesh.

```
# ; Create Test Mesh
import SyPy
hlev = SyPy.SyLevel()
hlev.OpenExisting()
# hlev.OpenSynthEyes()

def WriteVerts(msh, hgrid, vgrid):
    vc = 1
    for v in range(0, vgrid+1):
      for h in range(0, hgrid+1):
          tu = float(h) / float(hgrid)
          tv = float(v) / float(vgrid)
          pu = (tu - 0.5)*2   # raw image coordinates
          pv = (tv - 0.5)*2
          msh.Call("SetVtx", vc, [pu, pv, 0])
          msh.Call("SetVtxTexCo", vc, [tu, 1.0 - tv])
          vc += 1
    msh.Call("UpdateVtxs")
    msh.Call("UpdateTexCos")

def WriteFaces(msh, hgrid, vgrid):
    fc = 1
    stride = hgrid+1
    for v in range(0, vgrid):
      for h in range(0, hgrid):
          vb = v*stride + h + 1;  # Vertex numbers start at one
          msh.Call("SetFace", fc, [vb, vb+1, vb+stride+1])
          msh.Call("SetFaceShow", fc, 3)
          fc += 1
          msh.Call("SetFace", fc, [vb, vb+stride+1, vb+stride])
          msh.Call("SetFaceShow", fc, 6)
          fc += 1

hlev.Begin()
msh = hlev.CreateNew("MESH")
msh.nm = "My New Mesh"
msh.kind = "Test"
hgrid = 16
vgrid = 9
WriteVerts(msh, hgrid, vgrid)
WriteFaces(msh, hgrid, vgrid)
hlev.Accept("Create test mesh")
```

Blendermash Importer

Blendermash is a mashup of the Sizzle-based Blender exporter with SyPy3 to create a scene update importer that runs inside blender (which requires python 3). This demonstrates the advantages of SyPy's bring-your-own-python approach.

The nominal workflow is as follows:

- Track your scene in SynthEyes
- Export to blender 2.80+ using the normal SynthEyes blender exporter
- Via auto-run or Run Script in Blender, run the initial export.
- Work on the scene in blender.
- You update the tracking in SynthEyes
- At this point, you could export to Blender again using the "Update" option of the exporter, so that it can update the existing scene. However, you'd rather do the following....
- Open the blendermash.py script in blender's script editor.
- Click Run Script, and blendermash pulls the updated scene from SynthEyes (read setup info below.)
- From this point on, you can continue to change the SynthEyes tracking, and simply click Run Script again to repeatedly pull updates from SynthEyes into blender.

In order to do this, you must have done some preliminary setup work.

First, you must install SyPy3 into ***Blender's*** python installation. It can typically be found inside the blender installation folder; look for a folder named with the version number, ie a folder named 2.80 for blender 2.80. Inside that folder, see python/lib — that's where SyPy3 must go.

Second, you must arrange for SyPy3 in blender to be able to get the port and pin information for SynthEyes. You can do that a number of ways, probably using command line arguments for one or both applications and/or setting them in blendermash.

As a result, it's probably a little complex for most people to set up, but may be usable for others. At the least, it contains quite a bit of code showing how you can pull information out of SynthEyes and set up another application.